Les tableaux

Un tableau (Array) est une structure de données qui permet de stocker une liste d'éléments, généralement du même type, dans une seule variable. Chaque élément dans un tableau est identifié par un index (position). En programmation, le premier élément d'un tableau est souvent à l'index (position) 0.

Dans le cours, nous avons vu la nommenclature des variables. Nous préficerons le préfixe `a` pour les tableaux (du mot anglais "array") et mettrons le nom au pluriel car un tableau contient plusieurs éléments.

Déclaration et initialisation d'un tableau

Pour initialiser un tableau, on utilise des crochets `[]` et on sépare les éléments par des virgules.

```
let aEnnemis = ["gobelin", "orc", "dragon"];
```

Connaitre la taille d'un tableau

Pour connaître la taille (le nombre d'éléments) d'un tableau, on utilise la propriété `length`.

```
let nombreEnnemis = aEnnemis.length; // nombreEnnemis vaut 3
```

Accéder aux éléments d'un tableau

Pour accéder à un élément spécifique dans un tableau, on utilise son index entre crochets. Par exemple, pour accéder au premier élément (index 0) du tableau `aEnnemis` :

```
let premierEnnemi = aEnnemis[0]; // "gobelin"
```

Modifier les éléments d'un tableau

Pour modifier un élément dans un tableau, on utilise son index pour assigner une nouvelle valeur. Par exemple, pour changer le deuxième élément (index 1) du tableau `aEnnemis` :

```
aEnnemis[1] = "troll"; // Le tableau devient ["gobelin", "troll", "dragon"]
```

Pour accéder au dernier élément d'un tableau, on peut utiliser la propriété `length` du tableau pour obtenir sa taille et soustraire 1 pour obtenir l'index du dernier élément. Comme les index commencent à 0, le dernier élément est à l'index `length - 1`.

```
let bossEnnemi = aEnnemis[aEnnemis.length - 1]; // "dragon"
```

Ajouter des éléments à un tableau

Une fois le tableau créé, on peut ajouter des éléments en utilisant la méthode `push()`. Cela ajoute un élément à la fin du tableau.

```
aEnnemis.push("sorcier"); // Le tableau devient ["gobelin", "troll", "dragon",
"sorcier"]
```

Supprimer des éléments d'un tableau

Pour supprimer le dernier élément d'un tableau, on utilise la méthode `pop()`. Cela retire et retourne le dernier élément du tableau et on le stocke si besoin dans une variable.

```
let ennemiVaincu = aEnnemis.pop(); // "sorcier"
// Le tableau devient ["gobelin", "troll", "dragon"]
```

Pour supprimer un élément à une position spécifique, on peut utiliser la méthode `splice()`. Par exemple, pour supprimer le deuxième élément (index 1) du tableau `aEnnemis` :

```
aEnnemis.splice(1, 1); // Supprime "troll"
// Le tableau devient ["gobelin", "dragon"]
```

`Splice` peut aussi être utilisé pour ajouter des éléments à une position spécifique. Par exemple, pour ajouter "vampire" à l'index 1 :

```
aEnnemis.splice(1, 0, "vampire"); // Ajoute "vampire" à l'index 1, on retire 0
élément et on en ajoute 1 qui est "vampire"
// Le tableau devient ["gobelin", "vampire", "dragon"]
```

Parcourir un tableau

Pour parcourir un tableau et accéder à chaque élément, on peut utiliser une boucle `for`.

```
for (let i = 0; i < aEnnemis.length; i++) {
    console.log("Ennemi " + (i + 1) + ": " + aEnnemis[i]);
}
```

Exemple pratique : Dessiner des ennemis sur un Canvas

Dans cet exemple, nous allons utiliser deux tableaux pour stocker les positions X et Y de plusieurs ennemis, puis les dessiner sur un canvas HTML. Éventuellement, nous verrons comment utiliser les objets pour améliorer cette gestion au lieu de deux tableaux séparés.

```
// Tableau contenant les positions X des ennemis
let aNomsEnnemis = ["gobelin", "orc", "dragon", "sorcier"];
let aPositionsEnnemisX = [100, 250, 400, 550];
let aPositionsEnnemisY = [150, 200, 180, 160];

// Canvas et contexte
let canvas = document.getElementById("monCanvas");
let contexte = canvas.getContext("2d");

function boucleJeu(){
    dessinerEnnemis();
}

function dessinerEnnemis() {
    // Parcourir le tableau pour dessiner chaque ennemi
    for (let i = 0; i < aPositionsEnnemisX.length; i++) {
        // Dessiner un rectangle rouge représentant un ennemi
        contexte.fillStyle = "red";
        contexte.fillRect(aPositionsEnnemisX[i], aPositionsEnnemisY[i], 50, 50);
        // Ou drawImage si vous avez une image d'ennemi
        //let imageEnnemi = new Image();
        //imageEnnemi.src =  `images/${aNomsEnnemis[i]}.png`; // Assurez-vous
d'avoir une image d'ennemi correspondante
        //contexte.drawImage(imageEnnemi, aPositionsEnnemisX[i],
aPositionsEnnemisY[i]);
    }
}
```